#Course : DSC630

#Name : Tejashri Bhilare

#Week 10

#Assignmnet 10.2 : Time series modeling

Creating a recommender system using the small MovieLens dataset involves several steps,

1. Data preparation
2. building a user-item interaction matrix
3. and implementing a recommendation algorithm.

Step 1: Data Preparation

Download and Load the Data:

First, we will download the small MovieLens dataset from MovieLens.

then Load the required CSV files into pandas DataFrames.

In [3]:
```
import pandas as pd

# Load movies and ratings datasets

 # dataset Contains movie titles and genres
movies = pd.read_csv('C:/Users/Teju/Downloads/ml-latest-small/ml-latest-small/movies.csv')

# dataset Contains user ratings for movies
ratings = pd.read_csv('C:/Users/Teju/Downloads/ml-latest-small/ml-latest-small/ratings.csv')
```

In [4]:
```
print(movies.head())
print(ratings.head())
print(ratings.describe())
```

```
   movieId                               title  \
0        1                    Toy Story (1995)
1        2                      Jumanji (1995)
2        3             Grumpier Old Men (1995)
3        4            Waiting to Exhale (1995)
4        5  Father of the Bride Part II (1995)

                                        genres
0  Adventure|Animation|Children|Comedy|Fantasy
1                   Adventure|Children|Fantasy
2                               Comedy|Romance
3                         Comedy|Drama|Romance
4                                       Comedy
   userId  movieId  rating  timestamp
0       1        1     4.0  964982703
1       1        3     4.0  964981247
2       1        6     4.0  964982224
3       1       47     5.0  964983815
4       1       50     5.0  964982931
              userId        movieId         rating     timestamp
count  100836.000000  100836.000000  100836.000000  1.008360e+05
mean      326.127564   19435.295718       3.501557  1.205946e+09
std       182.618491   35530.987199       1.042529  2.162610e+08
min         1.000000       1.000000       0.500000  8.281246e+08
25%       177.000000    1199.000000       3.000000  1.019124e+09
50%       325.000000    2991.000000       3.500000  1.186087e+09
75%       477.000000    8122.000000       4.000000  1.435994e+09
max       610.000000  193609.000000       5.000000  1.537799e+09
```

Step 2: Create a User-Item Matrix

In [6]:
```
user_movie_matrix = ratings.pivot(index='userId',
                                  columns='movieId', values='rating').fillna(0)
```

Step 3: Choose a Recommendation Approach

We will Implement collaborative filtering based on user similarities.

We'll use cosine similarity to measure the similarity

between users based on their ratings.

In [7]:
```
from sklearn.metrics.pairwise import cosine_similarity

# Calculate the cosine similarity matrix
similarity_matrix = cosine_similarity(user_movie_matrix)
```

Here we will Create a DataFrame for Similarities.

And will Store the similarity scores in a DataFrame for easy access.

In [9]:
```
similarity_df = pd.DataFrame(similarity_matrix,
    index=user_movie_matrix.index, columns=user_movie_matrix.index)
```

Step 4: Build the Recommendation Function

we will Create a function that accepts a movie title

and recommends ten other movies based on the ratings of users who liked that movie.

In [11]:
```
def recommend_movies(movie_title, num_recommendations=10):
    # Get the movie ID from the title
    movie_id = movies[movies['title'] == movie_title]['movieId'].values[0]

    # Find users who rated this movie
    movie_ratings = ratings[ratings['movieId'] == movie_id]

    # Get similar users based on their ratings
    similar_users = similarity_df.loc[movie_ratings['userId']]

    # Aggregate ratings for all movies rated by similar users
    similar_users_ratings = ratings[ratings['userId'].isin(similar_users.index)]

    # Calculate the average ratings for movies rated by similar users
    movie_recommendations = similar_users_ratings.groupby
    ('movieId')['rating'].mean().reset_index()

    # Exclude the original movie and sort by ratings
    recommendations = movie_recommendations[movie_recommendations['movieId'] != movie_id]
    recommendations = recommendations.sort_values
    (by='rating', ascending=False).head(num_recommendations)

    # Merge with movie titles to get the recommended movie titles
    recommended_movies = recommendations.merge(movies, on='movieId')
    return recommended_movies[['title', 'rating']]
```

Step 5: User Input and Output

In [14]:
```
user_input = input("Enter a movie you like: ")
if user_input in movies['title'].values:
    recommendations = recommend_movies(user_input)
    print(f"\nRecommendations for '{user_input}':")
    print(recommendations)
else:
    print("Sorry, that movie is not in the dataset.")
```

```
Enter a movie you like: Toy Story (1995)

Recommendations for 'Toy Story (1995)':
                                          title  rating
0     Thin Line Between Love and Hate, A (1996)     5.0
1                          Particle Fever (2013)     5.0
2           No Direction Home: Bob Dylan (2005)     5.0
3                                Faster (2010)     5.0
4            The Girl with All the Gifts (2016)     5.0
5                 Ooops! Noah is Gone... (2015)     5.0
6                              Marwencol (2010)     5.0
7                    Down Argentine Way (1940)     5.0
8                      Girls About Town (1931)     5.0
9  Scooby-Doo! Curse of the Lake Monster (2010)     5.0
```